



Mini Vector Accelerator

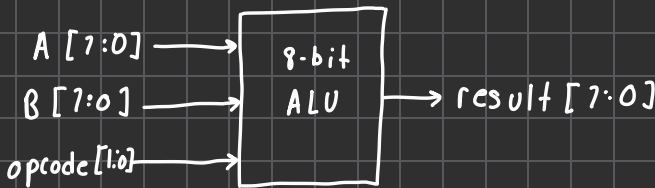
Aug. 26, 2026

Phase 1: 8-bit ALU

Purpose

- Combinational datapath performing an arithmetic or logical operation on two 8-bit operands
- 2-bit opcode selects which operation appears at the output

Block Level Specification



Interface

Signal	Width	opcode	result
a	8 bits	input	1st operand
b	8 bits	input	2nd operand
opcode	2 bits	input	select operation
result	8 bits	Output	operation result

Instruction / Operation Table

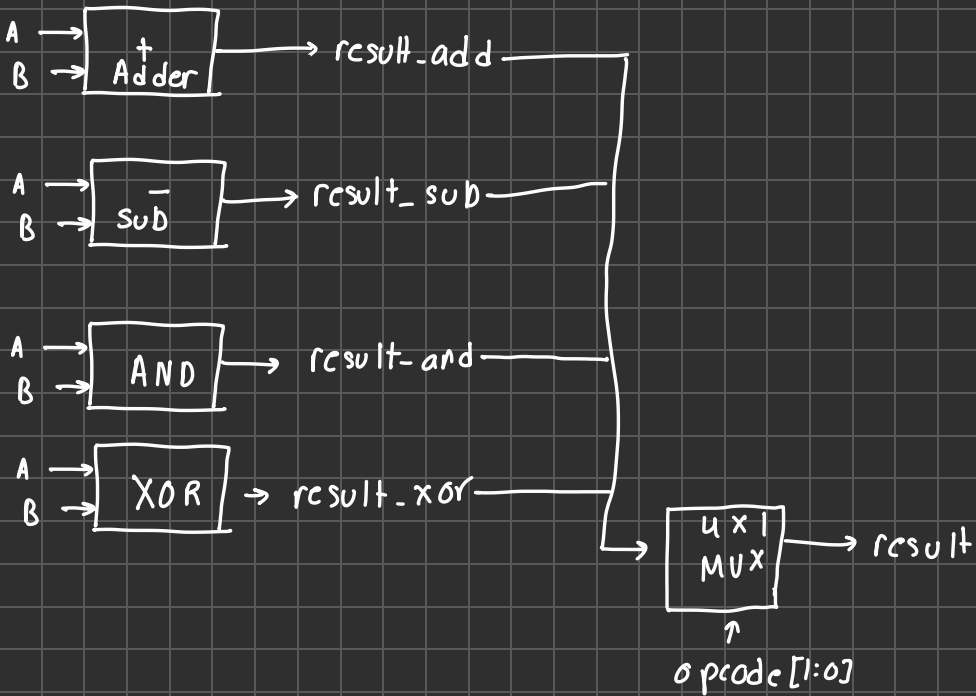
Opcode	Operation	Mathematical/Boolean Result
00	ADD	$result = a + b$
01	SUB	$result = a - b$
10	AND	$result = a \& b$
11	XOR	$result = a \oplus b$

Circuit Specifications

- result does not depend on past output values
↳ ∴ result is a combinational circuit

$$\text{result} = f(a, b, \text{opcode})$$

I am going to create 4 functional units where the final output is selected by a 4x1 Mux with the opcode.



Overflow & Bit Width

- If a & b are 8 bits long the maximum unsigned value is 255_{10} . So if we perform $a+b$ where both $a, b = 255_{10}$. The result becomes 510_{10} . This requires 9 bits when our result is 8.

∴ V1 intentionally ignores
carry-out and overflow

Test Case Verification Plan

Manual Testing

ADD Test Cases

Normal Addition

$$5 + 3 = 8$$

$$a = 0000\ 0101$$

$$b = 0000\ 0011$$

$$\text{opcode} = 00$$

$$\text{expected} = 0000\ 1000$$

8-bit Overflow/ Truncation

$$255 + 1 = 0$$

→ 8-bit truncation

$$a = 1111\ 1111$$

$$b = 0000\ 0001$$

$$\text{expected} = \cancel{1} 0000\ 0000$$

SUB Test Cases

Normal Operation

$$4 - 4 = 0$$

$$a = 0000\ 1001$$

$$b = 0000\ 0100$$

$$\text{opcode} = 01$$

$$\text{expected} = 0000\ 0101$$

Equal Operands

$$7 - 7 = 0$$

→ zero result expected

$$a = 0000\ 0111$$

$$b = 0000\ 0111$$

$$\text{expected} = 0000\ 0000$$

AND Test Cases

Normal Operation

$$a = 1100\ 1100$$

$$b = 1010\ 1010$$

$$\text{opcode} = 10$$

$$\text{expected} = 1000\ 1000$$

AND Buggy Behavior

$$a = 1111\ 1111$$

$$b = 0000\ 0000$$

$$\text{expected} = 0000\ 0000$$

XOR Test Cases

Normal Operation

a = 1100 1100

b = 1010 1010

opcode = 11

expected = 0110 0110

Equal Operands

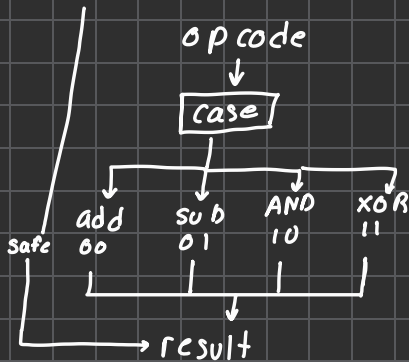
a = 1111 1111

b = 0000 0000

expected = 0000 0000

RTL Design Plan

① Operation Selection



* case(opcode) inside
always_comb to select
the ALU operation

* Give result a safe default

8-bit ALU Design Specification

Type: Combinational datapath

Inputs:

a [7:0]

b [7:0]

opcode [1:0]

Output:

result [7:0]

Operations:

* Depends on opcode bit value

00 → ADD

10 → AND

01 → SUB

11 → XOR

Implementation:

always_comb → case (opcode) → blocking assignments (=)

Storage:

None

Clock:

None

VI Limitations:

→ No carry flag

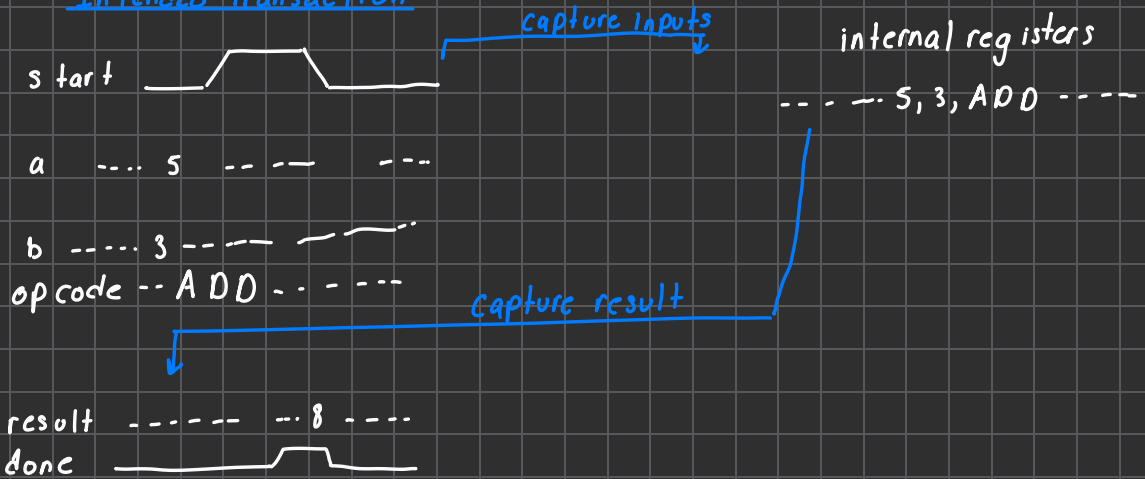
→ No zero flag

→ No overflow flag

→ 8-bit result only

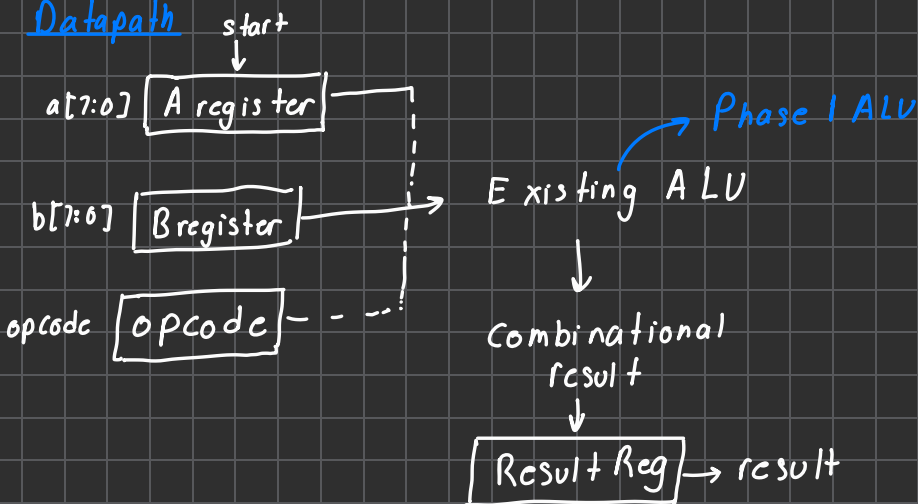
Phase 2: Registered ALU + start/done

Intended Transaction



* Start tells the hardware when the current input values should become an actual operation

Datapath



Registered ALU Verification Logic

* We tested the arithmetic functionality with alu.sv, so we need to ensure the registered ALU behaves as a small two-cycle hardware transaction

Cycle N

start = 1
↓
capture a, b, opcode
busy = 1

Between Cycles

→ ALU computes from registered inputs

Cycle N+1

→ result captures alu-result
→ done = 1
→ busy = 0

Cycle N+2

→ done returns to 0

Transaction Stages

* High-level hardware operation description

① Reset/Idle → ② Present Inputs → ③ Assert Start → ④ Capture inputs → posedge

→ ⑤ Execute ALU → ⑥ Capture Result → ⑦ Verify Result + Done → ⑧ Verify Done clears → next posedge

↙ between edges
↓ next posedge

⑨
→ Ready for Next Test

Test Vectors

→ Here we are aiming to test the circuit's registered wrapper & Transaction Timing

Test	Inputs	Oprodc	Expected result
ADD	a = 5, b = 3	00	8
SUB	a = 9, b = 4	01	5
AND	11001109 10101010	10	10001000
XOR	11001109 10101010	11	0110 0110

* We are expecting the same output
of the alu_tb.sv

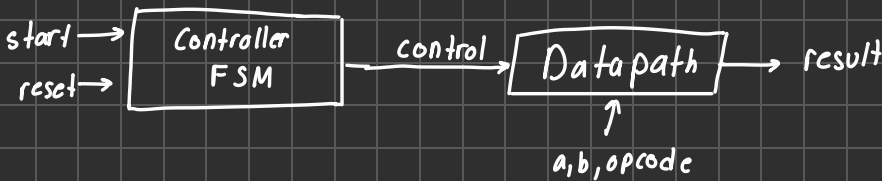
→ We are testing the trasaction procedure against these vectors

Phase 3 - Controller FSM Notes

① Purpose

- The controller FSM coordinates the sequence of operations performed by the datapath
 - ↳ Decides when
 - Input capture
 - ALU execution
 - Result storage
 - Module signals completion

High-Lvl Diagram



② Changes From Registered_ALU.sv

I decided to stop using the busy bit to represent the stages

bit = 0, 1

↳ Only two meaningful states

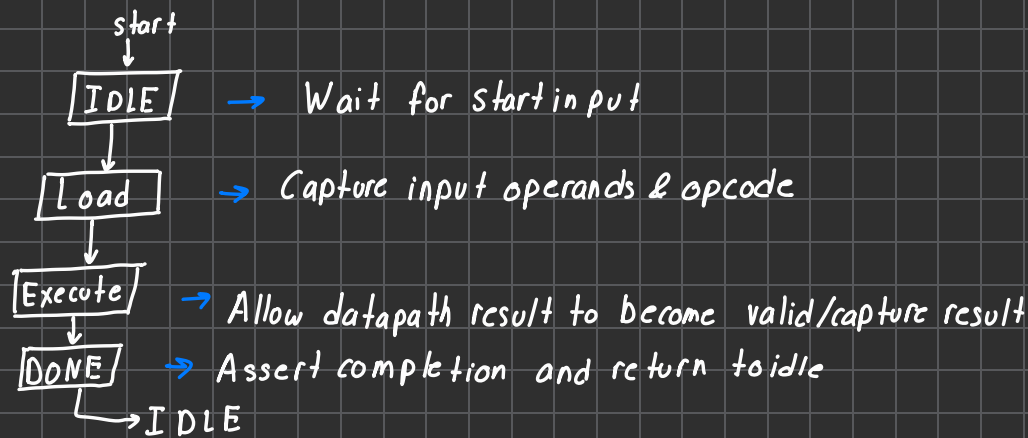
- Too fully realize the vector accelerator, I am going to need more states like

Idle → Load → Execute → Writeback → Done

↳ # of states >> 2

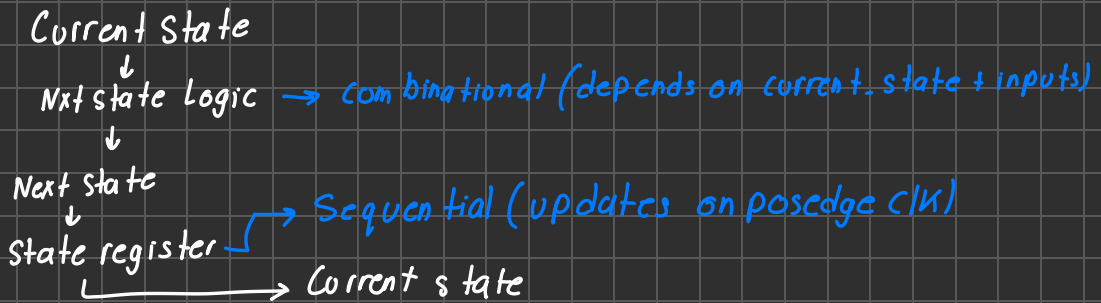
4 states → 2^2 → 2 bits for each state

③ FSM Definition



④ State Register vs Next-State Logic

Standard FSM Architecture



VHDL Logical Separation

always_ff
→ stores current-state

always_comb
→ calculates next state

⑤ State Transition Conditions

current state	Condition	Next state
IDLE	start = 0	IDLE
IDLE	start = 1	LOAD
LOAD	unconditional	Execute
EXECUTE	unconditional	DONE
DONE	unconditional	IDLE

⑥ State Based Datapath Actions

IDLE

done = 0
wait for start

LOAD

Capture:

a-reg $\leftarrow$ a

b-reg $\leftarrow$ a

opcode-reg $\leftarrow$ op code

EXECUTE

a-reg
b-reg $\rightarrow$ ALU $\rightarrow$ alu-result
opcode-reg

then

result $\leftarrow$ alu-result

DONE

set:

done = 1

$\rightarrow$ Go back
to Idle

⑦ Moore-style Output for first phase

* Outputs depend primarily on the current state

⑧ State Enum

```
typedef enum logic [10] {
```

 IDLE,

 LOAD,

 Execute,

 DONE

```
} State-6;
```

⑨ Reset Behavior

$\rightarrow$ result should set:

current-state = IDLE

result = 0

a-reg = 0

b-reg = 0

opcode-reg = 0

* Synchronous reset

reset = 1 + posedge clk

$\downarrow$
FSM enters
IDLE

⑩ Testbench Goals

Cycle N
→ start = 1

IDLE

Cycle N+1

LOAD

Cycle N+2

EXECUTE

Cycle N+3
→ done = 1

DONE

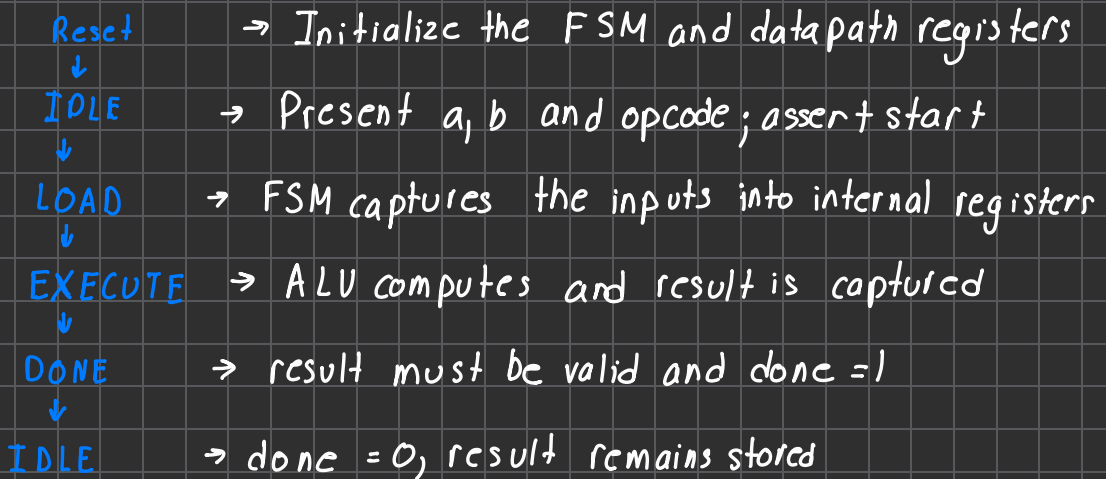
Cycle N+4
→ done = 0

IDLE

⑪ Test bench development

Purpose : Verify both the ALU datapath & FSM control sequence operate correctly across clock cycles

Transaction Sequence



Test bench Verification Goals

- Correct arithmetic/logic results
- Correct latency through FSM
- done only asserts during the DONE state
- done is a one-cycle pulse
- Multiple transactions can execute correctly

Proposed Testing Pattern

- Drive inputs → Assert start → Wait through FSM → Check results + Done
- Verifies done clears

Phase 4: Vector Datapath

Purpose: Extend our single element ALU datapath into a multi-lane architecture capable of operating on several data elements simultaneously

Scalar

$a \text{ OP } b \rightarrow$

Vector:

$A[0] \text{ OP } B[0] \rightarrow R[0]$
 $A[1] \text{ OP } B[1] \rightarrow R[1]$
 $A[2] \text{ OP } B[2] \rightarrow R[2]$
 $A[3] \text{ OP } B[3] \rightarrow R[3]$

Main Concept $\rightarrow$ Hardware parallelism

* Multiple ALU instances within FPGA operate during the same clock period

$\rightarrow$ differs from sequential software execution

How it connects to the Previous Phases

Combinational ALU $\rightarrow$ Provides the arithmetic unit used by each vector lane

Registered ALU $\rightarrow$ Introduced clocked storage and movement data through registers

Controller FSM $\rightarrow$ Controls when data is loaded, processed & completed

Vector Datapath $\rightarrow$ Replicates the arithmetic hardware so several elements can be processed simultaneously

Controller vs Datapath

Controller

$\rightarrow$ When data is loaded

Data Path

$\rightarrow$ What operations happen to the data

Accelerator Progression

Scalar ALU $\rightarrow$ Registered Datapath $\rightarrow$ FSM-Controlled Data path

$\rightarrow$ Parallel Vector Datapath $\rightarrow$ Multiply-Accumulate Units

$\downarrow$
We're here

$\rightarrow$ Dot Product $\rightarrow$ Matrix-Vector Multiplication

Test bench Development

Purpose: Verify that all four ALU lanes perform the selected operation correctly and in parallel

Combinational Circuit Test

- No clock
- No reset
- No FSM
- No stored state

Testing Sequence

Drive vector inputs → Select Opcode → Wait #1 for combinational logic to settle → Check All four Results

Testing Results

→ Each lane must behave like the original scalar ALU

↳ $A = [2, 4, 6, 8]$
 $B = [1, 3, 5, 7] \Rightarrow R = [3, 7, 11, 15]$
Opcode = ADD

AND & XOR Expected Results

AND

Lane 0

$A=2 = 0000\ 0010$
 $B=1 = 0000\ 0001$

 $0000\ 0000$

LANE 1

$A=4 = 0000\ 0100$
 $B=3 = 0000\ 0011$

 $= 0$

LANE 2

$A=6 = 0000\ 0110$
 $B=5 = 0000\ 0101$

 $0000\ 0100$

LANE 3

$A=8 = 0000\ 1000$
 $B=7 = 0000\ 0111$

 $= 0$

XOR

Lane 0

$A=2 = 0000\ 0010$
 $B=1 = 0000\ 0001$

 $0000\ 0011$

LANE 1

$A=4 = 0000\ 0100$
 $B=3 = 0000\ 0011$

 $0000\ 0111$

LANE 2

$A=6 = 0000\ 0110$
 $B=5 = 0000\ 0101$

 $0000\ 0011$

LANE 3

$A=8 = 0000\ 1000$
 $B=7 = 0000\ 0111$

 $0000\ 1111$

Phase 5: MAC/Dot-product Datapath

- ① Purpose: This stage moves our basic parallel ALU operations to Multiply-Accumulate (MAC) and dot-product operations

Dot-Product Path

→ Scalar operation that is incumbent in AI/ML

$$A = [2, 4, 6, 8]$$

$$B = [1, 3, 5, 7]$$

A.B Performs

$$= (2 \times 1) + (4 \times 3) + (6 \times 5) + (8 \times 7)$$

$$= 2 + 12 + 30 + 56 = 200$$

② Multiply-Accumulate (MAC)

$$* \text{ accumulator} = \text{accumulator} + (a \times b)$$

→ This phase moves the project from a general-purpose vector datapath to an AI/DSP-style hardware accelerator

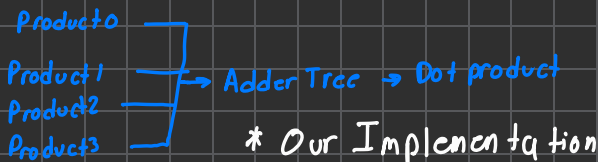
③ Sequential vs Parallel Architecture

Sequential MAC reuses one multiplier

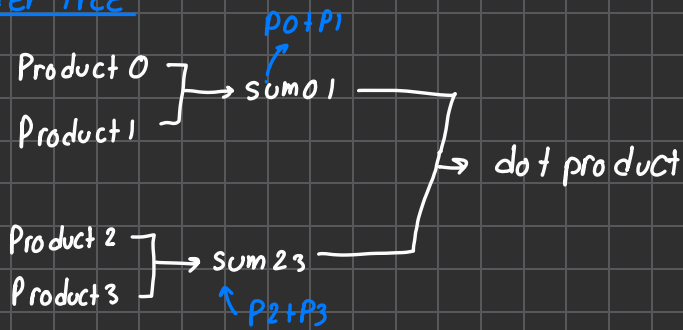
→ Less hardware (1 multiplier, 1 accumulator)

→ More clock cycles (↓ latency)

Parallel MAC Architecture makes multiple multipliers



④ Adder Tree



* Adder Trees help when designing larger datapaths bc the structure affects the hardware's critical path & timing performance

⑤ Bit Width Growth

* Multiplication & addition can require wider signals than the original operands

Max unsigned 8 bit value = 255

→ $A = 255, B = 255 \rightarrow A \times B = 65025$
↳ 16-bit product
(17 after addition)

For our 4 Lane Architecture

→ $4 \times 65025 = 260100$

Datapath Widths

Inputs → 8 bits

Outputs → 16 bits

Pair Sums → 17 bits

Final Result → 18 bits

Vector Dot-Product Testbench

Purpose:

Verify That:

- 1) Multiplies each corresponding vector element
- 2) Produces four independent 16-bit products
- 3) Adds the products through the adder tree
- 4) Produces the correct 18-bit values
- 5) Handles boundary values without losing bits

Test Sequence:

Drive A+B → Wait #1 → Calculate/Know expected dot product
→ Compare dot-product w/expected value → Load next test

Test 1 - Normal Dot Product

Goal: Verify Normal operations w/ small $\oplus$ values

Calculate

$$A = [2, 4, 6, 8]$$

$$B = [1, 3, 5, 7]$$

$$p_0 = 2 \times 1 = 2$$

$$p_1 = 4 \times 3 = 12$$

$$p_2 = 6 \times 5 = 30$$

$$p_3 = 8 \times 7 = 56$$

$$\text{sum}_01 = 2 + 12 = 14$$

$$\text{sum}_{23} = 30 + 56 = 86$$

$$\text{dot-product} = 14 + 86 = 100$$

* Verifies basic multiplication & adder-tree functionality

Test 2: Zero Inputs

Goal: Verify datapaths zero behavior

$$A = [0, 0, 0, 0]$$

$$B = [5, 10, 15, 20]$$

$$\rightarrow \text{dot-product} = 0$$

Test 3 - Larger / Mixed Values

$$A = [200, 40, 238, 198]$$

$$B = [25, 65, 242, 255]$$

$$p_0 = 5000$$

$$p_1 = 2600$$

$$p_2 = 57,596$$

$$p_3 = 50,490$$

$$\text{dot-product} = 115686$$

Test 4 - Maximum Value/Bit-Width Test

* Goal: Specifically verify the width calculations in the RTL

$$\text{Max_value} = 255$$

$$A = [255, 255, 255, 255]$$

$$255 \times 255 = 65025$$

$$B = [255, 255, 255, 255]$$

$$P_n = 130050$$

$$\text{dot} = 260100$$

Phase 6: Sequential MAC Unit

MAC (Multiply-Accumulate) unit performs

$$ACC_{new} = ACC_{old} + (a \times b)$$

Sequential MAC Procedure

$$A = [2, 4, 6, 8]$$

$$B = [1, 3, 5, 7]$$

$$ACC = 0$$

$$\text{Cycle 1: } 0 + (2 \times 1) = 2$$

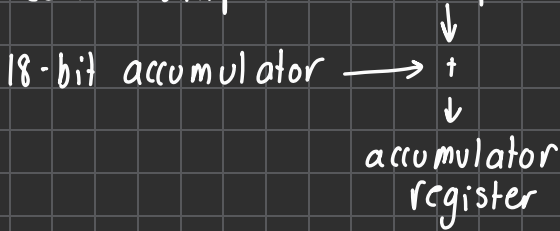
$$\text{Cycle 2: } 2 + (4 \times 3) = 14$$

$$\text{Cycle 3: } 14 + (6 \times 5) = 44$$

$$\text{Cycle 4: } 44 + (8 \times 7) = 100$$

Datapath

$a, b \rightarrow$ comb. multiplier $\rightarrow$ 16-bit product



Sequential MAC Testbench

Goal: Verify MAC behaviour around clockedges

Testing Sequence

Apply initial values $\rightarrow$ Wait for posedge clk

$\rightarrow \#1$ for non-blocking assignments to settle

$\rightarrow$ Check accumulator

Tests

① Reset

$\rightarrow ACC = 0$

② First Accumulation

$0 + (2 \times 1) = 2$

③ Second Accumulation

$2 + (4 \times 3) \rightarrow 14$

④ Disable MAC

$en = 0$

$\downarrow$

$ACC = 14$

⑤ Resume accumulation

$14 + (6 \times 5) = 44$

⑥ Final Accumulation

$44 + (8 \times 7) = 100$

⑦ Reset Again

$ACC \rightarrow 0$

Testbench verifies:

$\rightarrow$ Arithmetic correctness

$\rightarrow$ Accumulation across clockcycles

$\rightarrow$ Control behavior (reset/enable)

Phase 7: MAC Controller & Vector Sequencing

Main Goal:

→ Automates ($ACC \leftarrow ACC + (a \times b)$)

We aim to provide entire vectors:

$$A = [a_0, a_1, a_2, a_3]$$

$$B = [b_0, b_1, b_2, b_3]$$

pulse:

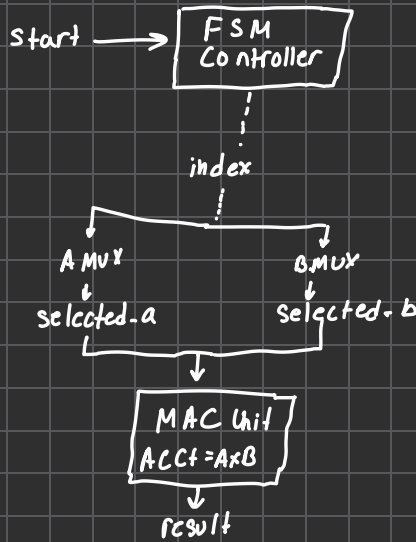
$$start = 1$$

and allow our hardware to automatically produce:

$$result = a_0b_0 + a_1b_1 + a_2b_2 + a_3b_3$$

Build Plan

FSM Controller
+
Index Controller
+
Multiplexers
+
MAC Unit



Index Counter

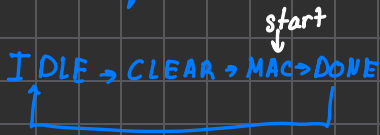
MAC → 2 bit Counter

* Transitions every clock cycle within the MAC

FSM States

States	Purpose
IDLE	wait for start
CLEAR	Reset accumulator and index
MAC	Process vector elements
DONE	Tell outside logic that result is valid

State Sequence



* We will be in "MAC" stage for a few clock cycles

Multiplexers

→ The index determines which input reaches the MAC

MAC Control

Existing MAC Architecture

- reset
- en

* We will generate those control signals from the controller

State Model

CLEAR state

- Reset MAC accumulator

MAC state

- $en = 1$
- accumulate $selected_a \times selected_b$

Other states

- $en = 0$ → hold accumulator

Example Execution

$$A = [2, 4, 6, 8] \\ B = [1, 3, 5, 7] \Rightarrow A \cdot B \Rightarrow 100$$

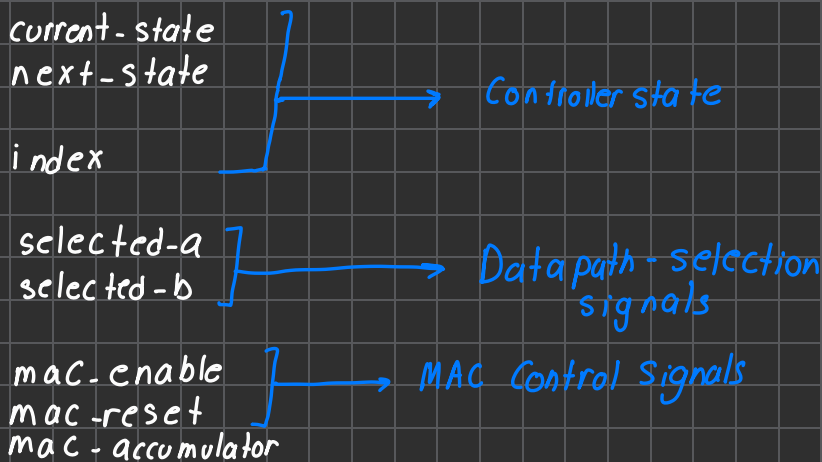
Hardware Timeline

clock	state	Index	Accumulator
0	IDLE	-	0
start ↓			
1	CLEAR	0	0
2	MAC	0	2
3	MAC	1	14
4	MAC	1	44
5	MAC	1	100
6	DONE	-	100

↳ done = 1

Needed Signals

We aim to make sure the motor controller has



Vector MAC Controller Testbench

Purpose:

Give MAC Controller
2 vectors



Pulse start



Controller sequences
elements automatically



Wait for done



Check Result

Phase 8: Signed INT8 Arithmetic

① Signed Arithmetic Addition

- Up until this point the accelerator has treated 8-bit values as unsigned
→ (0-255)
- However Neural Network calculations can contain negative values

Example

$$\begin{array}{l} \text{Inputs: } [2, -3, 4, -1] \\ \text{Weights: } [5, 2, -1, 6] \end{array} \Rightarrow 2(5) + (-3)(2) + 4(-1) + -1(6)$$
$$10 - 6 - 4 - 6 = -6$$

INT8

→ Signed 8-bit integer

* 8-bit Signed Range = -128 → +127

2's Complement

Conversion of -5

Start with +5
→ 0000 0101

⇒

Invert Bits
→ 1111 1010

⇒

Add 1

$$\begin{array}{r} 1111\ 1010 \\ + 1 \\ \hline 1111\ 1011 \end{array}$$

System Verilog

logic signed [7:0] a;

Signed Multiplication

assign product = a * b;
↓
signed 16-bit product

* Multiplication increases datapath width.
Never assume an 8-bit input results in another 8-bit signal

→ Multiply max values to find product width

Sign Extension

```
accumulator <= accumulator + {2'b0, product};
```

↓
zero-extension

↳ Only work for unsigned values

SystemVerilog Implementation

```
logic signed [15:0] product;  
logic signed [17:0] accumulator;
```

Signed bit replication

→ {2{product[15]}, product}

Making Accumulator Signed

* Signed arithmetic is a datapath-wide design decision

→ Every signal must carry a signed numerical value

Signed Literals

8'sd5 → 8-bit
→ signed
→ decimal 5

-8'sd8 → 8 bit
→ decimal -8
→ signed

Our Signed Data Signals

Datapath

→ a0 - a3

→ selected - a

→ product

→ result

→ b0 - b3

→ selected - b

→ accumulator

Control

→ Control signals remain unsigned
↳ do not do arithmetic operations